Q1: MCQ Revision (All Lectures)

** = supplementary, not in the lecture materials · Weeks 1 to 4 and iOS in full · Weeks 5 to 10 MCQ refresher + extra details not in the Q2 to Q5 notes

1. Introduction to mobile computing and networks (Week 1)

Mobile computing: technology that allows transmission of **data, voice and video** via a computer or any wireless-enabled device **without being connected to a fixed physical link**. The device and user are **free to move** while staying connected. Unlike desktop computing, it assumes **location, connectivity and resources can change at any time**.

Five core characteristics:

Characteristic	Meaning
Portability	Small, lightweight, battery-powered devices made for movement
Connectivity	Continuous or intermittent wireless access (cellular, Wi-Fi, Bluetooth)
Interactivity	Real-time, two-way communication between user, device and remote services
Individuality	Device personalised to one user's preferences, identity and context
Context-awareness	Apps sense and respond to location, motion, time and environment

Three components of a mobile computing system: Mobile hardware (smartphones, tablets, wearables with processors, sensors, radios), **mobile software** (OS and apps for constrained hardware), **mobile communication** (wireless infrastructure: cellular, Wi-Fi, satellite).

Network generations

Gen	Era	Peak speed	Key capability
1G	1980s	2 kbps	**Analog voice only**; no data; poor security and call quality
2G	1990s	64 kbps	**Digital** voice + **SMS**; introduced **GSM**; circuit-switched data
3G	2000s	~2 Mbps	**Packet-switched** data: mobile internet, video calls, early smartphones
4G / LTE	2010s	~100+ Mbps	**All-IP** network; mobile broadband, app ecosystems, streaming, cloud
5G	2020s	Multi-Gbps	**Ultra-low latency**, massive device density: IoT at scale, AR/VR, real-time apps

Why it matters to developers: apps must work across variable speeds (bandwidth assumptions); real-time apps need low latency; **offline-first design** (cache data, degrade gracefully); 5G device density enables large-scale **IoT**.

Cellular network architecture

Term	Meaning
Cell	A geographic area served by **one base station**
Base Station (BTS / eNodeB)	Transmits/receives radio signals to/from devices in its cell (BTS in 2G, eNodeB in 4G)
Mobile Switching Center (MSC)	Routes calls and data between cells and the wider network

Why cells exist: frequency reuse (same frequencies reused in **non-adjacent** cells), **capacity scaling** (smaller cells = more users in dense areas), **coverage efficiency** (lower power per cell = less interference and battery drain).

Mobility management, handoff and roaming

Mobility management: the network functions that **track a device's location** and **maintain connectivity as it moves**. Hard because it must work in real time, at scale, while saving battery and signalling capacity.

- **Location management:** network keeps registering **which cell/area** the device is in, so incoming calls/data can be routed.
- **Handoff management:** network **transfers an active connection** from one cell to another as the device moves.

Handoff (handover): transferring an **active call or data session** from one base station to another as a device moves between cells.

Hard handoff	Soft handoff
Old connection **dropped before** the new one is established ("break before make")	Device briefly **connected to both** cells during transition ("make before break")
Older / 2G-style systems	3G / 4G / 5G
Higher risk of dropped calls	Lower risk of dropped calls

Handoff steps: connected to Base Station A (strong) → moving, B's signal becomes stronger → network starts handoff to B → A released once B is active (soft). Apps must **tolerate short interruptions** rather than treat them as failures.

Roaming: a device staying connected and usable **outside its home network's coverage** by **using another provider's network**.

- **National roaming:** a different network in the **same country** (coverage gaps).
- **International roaming:** a **foreign carrier's** network when travelling abroad, governed by agreements between operators.
- Effects: **higher latency, different bandwidth, extra cost**. Good apps detect roaming and adapt (warn about data cost, low-bandwidth mode).

Mobile vs wireless vs ubiquitous computing

Term	Core idea	Example
Mobile computing	Computing that **moves with the user**; **portability** is the defining trait	Smartphone app on a train
Wireless computing	Connected **without cables**; need not be portable	Desktop PC on Wi-Fi
Ubiquitous (pervasive) computing	Computing **embedded invisibly** in everyday environments	Smart home sensors, wearables, ambient displays

Most mobile computing is also wireless, and it increasingly overlaps with ubiquitous computing through **IoT**.

Mobile app ecosystem (five parts)

Devices (hardware) · **Networks** (cellular, Wi-Fi) · **Platforms** (Android, iOS and their SDKs) · **App stores** (Google Play, Apple App Store: discovery, installation, monetization) · **Back-end services** (cloud APIs, databases, authentication). Developer's view: build for a platform → use device capabilities → connect to services. User's view: discover → install (grant permissions) → use.

Current trends: cross-platform frameworks (Flutter, React Native), 5G experiences (AR, cloud gaming), **on-device AI** (privacy, responsiveness), **privacy-first design** (GDPR, consent), **IoT convergence** (phone as controller).

Module facts: 2 h lecture + 3 h lab per week for 15 weeks. Assessment: Assignment 1 (individual) **10%**, Group project **30%**, Final exam **60%**.

2. Mobile operating systems (Week 2)

Mobile OS: system software that **manages hardware resources**, provides a **runtime environment** for apps and exposes **APIs**. Responsibilities: process management, memory management, security enforcement, hardware abstraction, UI services. Differs from desktop OS: strict **battery, memory and storage** constraints.

Android architecture (bottom to top)

Layer	Role
Linux Kernel (bottom)	Core services: **process and memory management, device drivers, security**
Hardware Abstraction Layer (HAL)	Standard interfaces so upper layers use hardware (camera, sensors, audio) **without device-specific details**
Android Runtime (ART) + Native Libraries	ART **compiles app bytecode to native instructions**; libraries such as **SQLite, WebKit, SSL**
Application Framework	Java/Kotlin APIs apps use: **Activity Manager, Window Manager, Content Providers, Notification Manager**, Views
Applications (top)	System apps (Phone, Settings) and third-party apps

iOS architecture (bottom to top)

Layer	Role
Core OS (bottom)	Kernel (**XNU**, based on Darwin/BSD), drivers; memory, security, threading, file system
Core Services	Foundation, networking, location, data management (**Core Data**)
Media	Graphics, audio, video, animation (Core Graphics, **Core Animation**, AVFoundation)
Cocoa Touch (top)	UI frameworks (**UIKit, SwiftUI**), gestures, notifications. **Developers work here most**

Android vs iOS

Aspect	Android	iOS
Kernel	Linux	XNU (Darwin/BSD)
Languages	**Kotlin**, Java	**Swift**, Objective-C
IDE	Android Studio (Windows, macOS, Linux)	**Xcode (macOS only)**
UI framework	XML layouts or Jetpack Compose	UIKit (Storyboards) or SwiftUI
Distribution	Google Play + other stores + **sideloading**	**App Store only**
Hardware vendors	Many (Samsung, Xiaomi…)	Apple only
Customisation	High (open-source)	Lower (closed, tightly controlled)
Fragmentation	High	Low
Screen unit	Activity / Fragment	ViewController

Other mobile OSs: HarmonyOS (Huawei, after losing Google services; mainly China), **KaiOS** (lightweight, for **feature phones** in emerging markets). **Windows Phone** and **BlackBerry OS** are discontinued. Android + iOS hold the overwhelming majority of the market.

Activity lifecycle

Lifecycle management exists because memory is limited and users switch apps, take calls and lock screens; the OS decides which apps stay, pause or die, and the app must save state and release resources.

States: Created → Started → Resumed (visible and interactive) → Paused (partly visible, not in focus, e.g. a dialog) → Stopped (not visible, e.g. switched apps) → Destroyed.

Callback	When / what to do
onCreate()	Called **once** when created; initialise UI (`setContentView`) and data
onStart()	Activity becomes **visible**
onResume()	Activity becomes **interactive** (in foreground)
onPause()	**Loses focus**; save lightweight state, pause actions (e.g. video/music)
onStop()	**No longer visible**; release resources not needed while hidden
onDestroy()	Final clean-up before removal from memory
onRestart() **	Called before onStart() when coming back from Stopped

Rotation problem: rotating the screen **destroys and recreates** the Activity (onDestroy → onCreate), so unsaved form data is lost. **Fix:** save state in **`onSaveInstanceState()`** and restore in onCreate, or use a lifecycle-aware **ViewModel** (survives configuration changes).

Process model, sandbox and permissions

- **Process model:** each app runs in **its own Linux process with a unique user ID (UID)**. Priority levels: **foreground, visible, service, background, empty**; lowest priority is killed first when memory is needed. A background app can be killed anytime, so persist important state.
- **Sandbox:** each app runs in an **isolated environment**; Android enforces it with a **unique Linux UID per app** (kernel blocks access to other apps' files/memory). iOS uses a kernel sandbox with **entitlements**. Limits damage from malicious or buggy apps.

Install-time permissions	Runtime permissions
Low-risk (e.g. **internet**)	**Sensitive** (camera, location, contacts)
Granted automatically at install	Requested **while the app runs**; user approves or denies
User can only refuse to install	User can **revoke anytime** in settings

Pre-Android **6.0**: install-time only (accept all or don't install). Android 6.0+ and iOS: runtime requests. **Principle of least privilege:** request only needed permissions, when needed. If denied, the app must **not crash**; explain why and offer an alternative.

Fragmentation and compatibility

Fragmentation: the wide variety of **Android OS versions, screen sizes and hardware** in use at the same time. **Cause:** updates depend on **manufacturers and carriers**. Developers must test across versions and screens. iOS has little fragmentation because Apple controls hardware and software.

Tool	Purpose
minSdkVersion	**Oldest** Android version the app supports
targetSdkVersion	Version the app is **optimised and tested for**
AndroidX libraries	Backward-compatible APIs across OS versions
Responsive layouts	Adapt to screen sizes and densities

Android Studio and project structure

Term	Meaning
Android Studio	Official IDE, by Google, built on **JetBrains IntelliJ IDEA**
Android SDK	Libraries, platform tools, APIs to build, debug and run apps; includes **adb** (Android Debug Bridge) and the Emulator. Separate from the IDE
AVD / Emulator	Simulated Android device = hardware profile + system image
Gradle	**Build system**: compiles code, manages dependencies, packages the app (APK)
Kotlin	Google's **preferred** Android language **since 2019**; by JetBrains; runs on the JVM; concise, **null-safe**, **fully interoperable with Java**

Project part	Contains
manifests/AndroidManifest.xml	Declares app **components, permissions and metadata** (and the launcher Activity)
java/ (or kotlin/)	Source code: Activities, Fragments, classes
res/	Resources: **layout XML** (`res/layout`), images (`res/drawable`), strings and colours (`res/values`)
build.gradle (project and module)	Dependencies, SDK versions, build settings

New project: **Empty Activity** template → `MainActivity.kt` (with `onCreate()`) + `activity_main.xml` (a TextView "Hello World!"). Run (▶) builds an APK, installs and launches it; Debug (bug icon) also attaches the debugger. Typical Minimum SDK in the lab: **API 24 (Android 7.0)**.

3. Mobile development platforms (Week 3)

Approach	Definition	Advantages	Limitations	Typical use
Native	Built for **one platform** using its official language and SDK (Kotlin/Java for Android, Swift for iOS)	**Best performance**; full access to all APIs and hardware **on day one**	**Separate codebases/teams** for each platform: highest cost and time	Games, AR/VR, deep hardware integration
Hybrid	A **web app (HTML/CSS/JS)** wrapped in a **native shell**, rendered through a **WebView**	Single codebase; reuses web skills and code; cheapest, fastest	Performance and UI feel limited by WebView; native features need plugins	Content apps, internal business tools, MVPs
Cross-platform	**One codebase** in a framework that compiles to / renders **native UI** on each platform	**Near-native** performance and look; shares most code	Sometimes needs platform-specific code for new features; extra abstraction layer	**Most commercial apps today**

Dimension	Native	Hybrid	Cross-platform
Performance	**Best**	Lower	Near-native
Development cost	**Highest** (2× teams)	**Lowest**	Moderate
Time-to-market	**Slowest**	**Fastest**	Fast
Maintainability	Two codebases	One codebase	One codebase
New OS features	**Immediate**	Delayed	Often delayed

Tools: Android Studio (IntelliJ-based; Kotlin preferred, Java; layout editor, emulator, profiler, Gradle). **Xcode** (Apple, **macOS only**; Swift preferred, Objective-C; Interface Builder, Simulator, Instruments).

Flutter	React Native
By **Google**	By **Meta** (Facebook)
Language: **Dart**	Language: **JavaScript/TypeScript** with React
Renders its own widgets directly (not native OS components): very consistent look across platforms	Renders **actual native UI components**: more platform-authentic feel

Hybrid tools **: Apache Cordova, Ionic/Capacitor. Game engines: **Unity / Unreal** (compile natively).

Decision framework (5 questions): 1. What must the app do? (heavy graphics/AR/hardware → native) 2. Budget and timeline? (tight → hybrid or cross-platform) 3. Team skills? (web devs → hybrid; JS/React team → React Native) 4. How long maintained? (long-lived → single cross-platform codebase) 5. Need day-one new OS features? (yes → native).

Worked examples: 3D game studio → **native** (or Unity/Unreal). Startup MVP, both platforms in 8 weeks, 2 devs, small budget → **cross-platform**. Internal expense tool by web team → **hybrid**.

Environment setup steps: choose approach → install IDE + accept SDK licences → configure an AVD or connect a device with USB debugging → create a project from a template to verify.

4. Mobile application architecture (Week 4)

Software architecture: the high-level structure of a system: how components are organised and interact. Without it, UI, business logic and data access get mixed (hard to test, debug, extend).

MVC vs MVVM

MVC	MVVM
Model: data + business logic	**Model:** data + business logic
View: displays data, captures input	**View: observes** the ViewModel and updates automatically
Controller: takes input from View, updates Model, refreshes View	**ViewModel:** holds UI state and logic, exposes observable data, **no reference to the View**
In Android the **Activity is often both View and Controller** → “massive Activity”	Android **ViewModel survives rotation**; uses **LiveData / StateFlow**

Aspect	MVC	MVVM
View-logic coupling	Tighter	Looser
Testability	Harder	**Easier** (ViewModel tested without UI)
Survives rotation	Not by default	**Yes** (ViewModel)
Learning curve	Lower	Slightly higher
Best for	Small, simple apps	Apps expected to grow

Four core Android components

Component	Role	Example
Activity	**A single screen with a UI**	Shows the message to the user
Service	Runs operations **in the background without a UI**	Playing music, downloading/syncing data
Broadcast Receiver	Listens/responds to **system-wide or app-wide announcements**	Battery low, network changed, new message arrived
Content Provider	Manages **shared access to structured data** between apps	Contacts shared with other apps

Messaging example: Broadcast Receiver detects the incoming message → Service downloads it in the background → Content Provider stores it → Activity displays it. Background music player → **Service**. All four are declared in **AndroidManifest.xml**.

Fragments, state and data binding

- **Fragment:** reusable portion of UI **inside an Activity** (multi-pane, tabs). Own callbacks **onCreateView(), onViewCreated(), onDestroyView()**. Its view can be destroyed and recreated **independently** of the Fragment object

(source of bugs); use a ViewModel scoped to the Fragment.

- **State management:** tracking the data that decides what the UI shows. **UI state** (on screen now, e.g. spinner visible) vs **persistent state** (must survive, e.g. login status, drafts). Harder on mobile because of rotation, backgrounding and low-memory kills.
- **Data binding:** automatically **synchronises UI elements with data**; the View observes data and updates itself, removing manual update code. It is what makes MVVM practical.
- **Separation of concerns:** each part has **one responsibility**. **UI layer** (display, forward actions, no business logic) → **Logic layer (ViewModel)** → **Data layer (Model / Repository)**: local DB, network or cache behind one interface. Each layer can be changed or tested independently.

MVVM multi-screen steps: define Model (data class) → create ViewModel exposing data → build two Activities/Fragments observing it → navigate with Intents or the Navigation component; both screens stay in sync.

5. iOS application development (Week 11)

- **iOS ecosystem: closed, controlled end to end by Apple** (devices, iOS, Xcode, Swift, UI frameworks, App Store). Fewer variations to support but less developer freedom. Theory only (needs macOS).
- Runs only on **iPhone, iPad, iPod Touch**. Less fragmentation, simpler testing, consistent UI; no low-cost device tier.
- **Xcode** (macOS only) for iOS, iPadOS, macOS, watchOS, tvOS. **Interface Builder** (Storyboards, SwiftUI preview), **iOS Simulator**, **Instruments** (profiler), **LLDB** (debugger).

iOS project file	Role
AppDelegate.swift	App-level events: **launch and termination** (like Android's Application class); marked `@main`
SceneDelegate.swift	UI scenes, **multi-window**; sets the first ViewController
ViewController.swift	Controls **one screen** (= Android **Activity**)
Main.storyboard	Visual layout of screens and navigation
Assets.xcassets	Images, icons, colours
Info.plist	Configuration: name, version, **permissions**, capabilities

ViewController lifecycle: `viewDidLoad()` (once, setup) → `viewWillAppear()` (every time before visible) → `viewDidAppear()` → `viewWillDisappear()` → `viewDidDisappear()` (stop timers). Roughly parallels onCreate, onStart, onResume, onPause, onStop.

Swift: Apple's modern, **safe, fast, readable** language that replaced Objective-C; strict compile-time checks; **type inference**. (Kotlin plays the same role on Android.)

```
var score = 90                  // var = can change
let maxScore = 100              // let = constant
let name = "Kamal"              // type inferred (String)
let age: Int = 21               // explicit type
func sum(a: Int, b: Int) -> Int { a + b }
var email: String?              // optional: a value or nil
if let userEmail = email { print(userEmail) }   // optional binding (safe unwrap)
let shown = email ?? "guest@site.com"           // nil-coalescing default
// email!  = force unwrap: crashes if nil (avoid)
class User { var username: String
             init(username: String) { self.username = username } }
```

Optionals are Swift's answer to null (like Kotlin nullable types `String?`).

UIKit (Storyboards)	SwiftUI
UI built **visually** in Storyboards (UILabel, UITextField, UIButton)	UI written **declaratively in Swift code**
State updated **manually**	State changes **redraw UI automatically** (`@State`)
Mature; runs most shipping apps; more complex	Apple's direction for new apps; less boilerplate

Storyboard connections: IBOutlet connects a UI element to a property (`@IBOutlet weak var usernameField: UITextField!`); **IBAction** connects a user action (tap) to a method; **Segue** is a visual link to the next screen (`performSegue(withIdentifier: "homeSegue", sender: self)`). SwiftUI uses `VStack`, `TextField("Username", text: $username)`, `Button("Login") { }`.

App Store: the **only** channel. Needs a **paid Apple Developer Account (USD 99/year)** and a **digitally signed build**; every submission passes **automated and human review (typically 1 to 3 days)**, then approved or rejected. Review enforces security (sandboxed, signed), privacy (declare data use, runtime permission) and consistency (**Human Interface Guidelines**).

6. Kotlin basics for Android **

Kotlin	Meaning
`val x = 5`	Read-only (cannot reassign)
`var y = 5`	Mutable (can change)
`var name: String? = null`	`?` = **nullable** type
`name?.length`	Safe call: returns null instead of crashing
`name ?: "Guest"`	Elvis operator: default if null
`name!!`	Force non-null: throws NullPointerException if null
`"Hello $name"` / `"${a + b}"`	String templates
`fun add(a: Int, b: Int): Int = a + b`	Function
`when (x) { 1 -> ...; else -> ... }`	Like switch
`listOf(...)` / `mutableListOf(...)` / `arrayListOf(...)`	Read-only list / changeable lists
`data class User(val name: String)`	Data-holding class (equals, toString, copy)
`object : Interface { }`	Anonymous object; `lateinit var` = initialise later

7. MCQ refresher: Weeks 5 to 10 (one-line essentials)

UI and navigation (Weeks 5 to 6)

- Layout = **ViewGroup**; widget = **View**. `match_parent` fills parent, `wrap_content` fits content. **dp** for sizes, **sp** for text.
- **ConstraintLayout** recommended (flat, fastest); **LinearLayout** one row/column; **FrameLayout** stacks (fragment container); **GridLayout** calculator; Relative/Table = legacy.
- **Text boxes: EditText** = input (`hint`, `inputType="textPassword"`), read with `.text.toString()`; **TextView** = display only; Material: **TextInputLayout + TextInputEditText** (floating label, error, password toggle).
- **`findViewById<Button>(R.id.btnLogin)`** gets a view by ID **after `setContentView()`**; IDs made with `@+id/`; **R** = auto-generated resource class.
- Event flow: User Action → Event → Listener → Code → UI Feedback; `setOnClickListener { }`.
- **Toast:** `Toast.makeText(this, "msg", Toast.LENGTH_SHORT).show()` (no `.show()` = nothing shown).
- **Intent** = message to start a component. **Explicit** names the class `Intent(this, Home::class.java)`; implicit describes an action. `putExtra("KEY", v)` / `getStringExtra("KEY")`; `startActivity()` opens, `finish()` closes.
- **Bottom nav** = 3 to 5 destinations; **Tabs** = related views of same data; **Drawer** = many, slides from left; **Toolbar** = title + actions; **AlertDialog** = confirm destructive action.
- **Fragment** = part of UI inside an Activity with own lifecycle; swapped with `supportFragmentManager...replace().commit()`.
- **RecyclerView** = RecyclerView + **Adapter** + **ViewHolder** + **LayoutManager**; `onCreateViewHolder` (inflate), `onBindViewHolder` (fill data), `getItemCount`; no LayoutManager = blank. **ScrollView** = one direct child. **Adapter** = bridge from data to list rows.
- **Material Design** (Google, 2014); **M3 / Material You** (2021, Android 12, **dynamic color**). Principles: surface/elevation, bold design, meaningful motion (150 to 300 ms). One **FAB** per screen; buttons Filled > Elevated > Outlined > Text.
- **UI/UX:** wireframe (low-fidelity) → prototype (clickable, Figma) → usability test. **48dp** touch target, **12sp** min text, **4.5:1** contrast, primary actions low (reach zone). Nielsen heuristics.

Web services (Week 7)

- Every web service is an API, not vice versa. **SOAP** = XML-only protocol, can be stateful; **REST** = HTTP style, JSON, **stateless**.
- GET read, POST create, PUT replace, PATCH partial, DELETE remove. 200 OK, 201 Created, 404 Not Found, 500 server error.
- JSON `{ }` object, `[ ]` array; smaller/faster than XML. Parsers: **org.json** (built-in), **Gson**, **Moshi**, **XmlPullParser**, SAX, DOM. Clients: **Retrofit** (best), **OkHttp**, **Volley** (older).

- **Postman** tests requests without app code; **mock API** = fake data (json-server, MockAPI.io, Postman Mock Servers). Need **INTERNET** permission; no network on main thread.

Location and sensors (Week 8)

- **Fused Location Provider** (Play services, combines signals) vs **LocationManager**. Data: latitude, longitude, accuracy, timestamp.
- **COARSE** approximate, **FINE** precise; check → request → grant/deny at **runtime**. Maps **displays**, Location API **provides**. Geofence: **ENTER, EXIT, DWELL**.
- **SensorManager, Sensor, SensorEvent, SensorEventListener**; **accelerometer** = linear movement/shake, **gyroscope** = rotation, **proximity** = near/far; unregister to save battery; camera via **CameraX**. Gestures: tap, long press, swipe/fling, pinch (**GestureDetector**).

Databases (Week 9)

- **SharedPreferences** key-value; **SQLite** embedded relational; **Room** = Jetpack layer over SQLite (**Entity / DAO / Database**, compile-time SQL check); **Firestore** = NoSQL **collection → document → field**; Realtime DB = JSON tree.
- **SQLiteOpenHelper**: `onCreate` creates tables, `onUpgrade` on version change; `ContentValues` + `insert()`; `rawQuery()` → **Cursor**; `?` prevents SQL injection.
- **Data class** auto-generates equals, hashCode, toString, copy. DAO methods are `suspend` (off main thread).
- Sync: online-only, periodic, real-time, **offline-first**; conflicts: last-write-wins, timestamp/version, field merge. UI → ViewModel → **Repository** → Room / Firebase.

Testing (Week 10)

- **Unit** = JVM, fast, `src/test`, **JUnit** + **Mockito**; **UI/instrumented** = device, `src/androidTest`, **Espresso** (`onView → perform → check`).
- `assertEquals(expected, actual)`, `assertTrue`, `assertNull`, `assertThrows`; Mockito `` `when`().thenReturn() `` stubs, `verify()` checks calls. AAA = Arrange, Act, Assert.
- **Emulator**: fast, many configs, no hardware; **real device**: true performance, battery, sensors, before release. **AVD** = system image + hardware profile (AVD/Device Manager).
- **Logcat** logs (V, D, I, W, E); **Debugger** breakpoints, Step Over/Into/Out; **Layout Inspector** view tree; **Profiler** CPU, memory, network.

8. Weeks 5 to 10: extra facts NOT in the Q2 to Q5 notes

Everything else from Weeks 5 to 10 is in the Q2 to Q5 PDFs. This section only adds the smaller details from those lectures, guides and practical sheets that the Q2 to Q5 notes left out.

Week 5: Layouts guide

- Every layout is a **ViewGroup**, which **measures and arranges** its children. Choosing the right layout affects **rendering performance, maintainability and adaptability** to screen sizes.
- Since **2020**, Google recommends **Jetpack Compose** (declarative Kotlin UI, replaces XML) for new UI. XML layouts are the "**View system**", still the base of most existing apps.
- **LinearLayout:** children placed **in the order they appear in the XML**. `layout_weight` only shares the **remaining** space after measuring declared sizes.
- **ConstraintLayout:** default layout in Android Studio templates **since 2016**; constraints are solved **together as one system** (single pass). Extra features: **chains, guidelines, barriers, dimension ratios**; can replace LinearLayout, RelativeLayout and some GridLayout uses in one **flat** hierarchy.
- **RelativeLayout:** needs **two measure/layout passes** (slower). Attributes: `layout_below`, `layout_centerInParent` (used in Practical 1's Hello Mobile World).
- **FrameLayout:** later children are drawn **on top** (z-order). Hosts one Fragment at a time via **FragmentContainerView**; used for caption overlays and loading spinners.
- **TableLayout:** rows declared with **`<TableRow>`**; column width = widest cell; **`stretchColumns`** (expand) and **`shrinkColumns`** (compress); **no cell spanning**.
- **GridLayout:** introduced in **API level 14 (Android 4.0)**; uses `rowCount`/`columnCount`; spanning with **`layout_rowSpan`** / **`layout_columnSpan`**.
- Recommended learning path: LinearLayout first, then ConstraintLayout (production standard); evaluate Compose for new projects.

Week 5: Material Design guide

- **Material Design objectives:** consistency, usability, accessibility, responsive layouts, standard design principles. Foundations: **colour system, typography scale, iconography, elevation**.
- **Elevation is measured in dp.** In **Material 3**, elevation is also shown with a **tonal surface tint**, not only shadows (useful in **dark theme** where shadows are hard to see). App bar elevation becomes **0** when scrolled to top in M3; FAB rises higher while **pressed or dragged**.
- **M3 colour:** each role is a **tonal palette of 13 tonal steps**, giving correct light and dark values automatically and supporting dynamic colour.
- Bold design favours a **small, purposeful colour palette used consistently** over many variations.
- Text field has **three states**: floating label at rest, **active** (highlighted outline when focused), **error** (red outline + inline message). Text fields also support **helper text**.
- Lecture slide wording: principle 1 is **“Material is Metaphorical”** (components behave like physical objects). Slide 6: a traditional EditText **has no built-in error handling** and **no visual feedback**.

Week 5 to 6: RecyclerView guide extras

- RecyclerView creates elements **only as they are needed**; benefits stated: **better performance, smoother responsiveness, lower power consumption**.
- Official guide’s **3 implementation steps:** 1. **plan the layout** (pick a LayoutManager), 2. **design the item and ViewHolder**, 3. **define the Adapter**.
- All LayoutManagers extend the abstract **`RecyclerView.LayoutManager`** class. LayoutManager can also be set in XML with **`app:layoutManager`**.
- **`tools:listitem="@layout/..."`** shows sample rows in Android Studio’s **design preview** only (no effect at runtime).
- **Edge-to-edge:** from **Android 15 (API 35)** apps are edge-to-edge by default, so first/last rows can hide under the status/navigation bars. Fix: **`enableEdgeToEdge()`**, apply insets with **`ViewCompat.setOnApplyWindowInsetsListener`**, and **`android:clipToPadding="false"`**.
- `notifyDataSetChanged()` **disables item animations**. ListView reuses views manually through **`convertView`**.

Week 6: UI/UX flow activity details

- Prototype interfaces are **embedded in Android Studio** as matching layouts and screens (Activity or Fragment). Figma frames sized **360 × 800** (typical Android phone).
- Source of the principles: **Android Developers, “Design for Android”** and accessibility guidance.
- QuickCart screens: **Checkout, Settings, Sign Up**. Deliverables: Activity 1 notes (3 to 5 issues per screen), Figma file, Android Studio project.

Week 7: Web services (small details)

- Web service features include **self-contained requests** (each carries everything the server needs).
- SOAP is common in **banking and enterprise** systems.
- In the Weather App practical, students must identify for the chosen API: **API name, endpoint, HTTP method, required parameters, API key requirement, JSON response structure**, and must not use **hard-coded** values.

Week 8: Location and sensors (lab sheet details)

- **Group activity apps:** 1 “Where Am I?” (one-time fetch: lat, lng, **accuracy, timestamp**), 2 Zone checker (**`distanceTo()`** vs radius, without the Geofencing API), 3 Distance tracker (two points), 4 Speed snapshot (**`LocationRequest`**, continuous updates, **`getSpeed()`** in m/s, stop updates in **onPause()/onStop()**), 5 Geofence alert (**GeofencingClient, GeofencingRequest, PendingIntent, BroadcastReceiver**, **`INITIAL_TRIGGER`**).
- **Geocoder** = **reverse-geocoding**: converts coordinates into a readable **address**.
- Test location on an emulator with **Extended Controls → mock location** (and route simulation for movement).
- Geofence alerts can be shown as a **Toast or notification**. Background location may be needed for geofencing on newer Android versions.
- Smartphone sensors also include **light** and **magnetic-field** sensors. Accelerometer values are affected by **movement and gravity**; values arrive in **`SensorEvent.values[]`**.

Week 9: Database lab details

- Lab projects use **Empty Views Activity**, Kotlin, **View Binding enabled**, min **API 24**.
- Room setup needs the **Room compiler, Room KTX** and the **kapt/ksp** plugin. Build the database **once** (in an Application class or `object`).

- Firestore setup: Firebase console → register app by **applicationId** → download **google-services.json** into `app/` → Google Services Gradle plugin + **Firebase BoM** → create Firestore in **test mode** (development), then tighten **Security Rules** (e.g. only authenticated users can write).
- **Offline persistence** in Firestore keeps previously loaded data visible in **Airplane Mode**.
- Capstone sync ideas: add a **`syncStatus`** field (**SYNCED, PENDING, CONFLICT**) or a pending flag/timestamp; trigger sync with a **connectivity listener or WorkManager** instead of constant polling; schema changes need a **Migration object** instead of destructive recreation.
- Room compile-time validation can be shown by **deliberately breaking an `@Query` string** (the build fails).

Week 10: Testing worksheet details

- A failing test shows a **failure message in the test report**; tests turning green after a fix confirms the regression is solved.
- Worksheet AVD: **API 34, 6.1-inch, 2 GB RAM**. If no physical device: use a **second AVD** with a different API level and screen size.
- **Conditional breakpoint** example condition: `email.isEmpty()`.

9. Practical 1 (Week 1 lab) and cross-platform practicals

Android Studio setup facts (Practical 1)

- Workflow: **Developer → Android Studio → Kotlin code → Build → Emulator → Test**. Emulator relationship: **Computer (host) → Emulator → Android OS**.
- **SDK Manager** (Tools → SDK Manager): **SDK Platforms** tab (e.g. API 34/35) and **SDK Tools** tab (**Build-Tools, Platform-Tools, Emulator, Command-line Tools**). Verify with **`adb --version`**. SDK default path `C:\Users\<Name>\AppData\Local\Android\Sdk`.
- Installer components: Android Studio, **Android Virtual Device**, **Intel HAXM** (performance, Intel CPUs).
- **Device Manager** (Tools → Device Manager) → Create Device → hardware (e.g. Pixel 7) → system image → Finish → Play ▶.
- IDE panels: **Project Explorer**, **Editor** (red squiggle = error, yellow = warning, **Ctrl+Space** = code completion), **toolbar** (device drop-down, **Run ▶ Shift+F10**, **Debug Shift+F9**, Stop ■), **Device Manager**, **Logcat**, **Build** tab (Gradle progress and errors; shows **BUILD SUCCESSFUL**), emulator controls (Power, Volume, Rotate, Screenshot, **Extended controls** ⋮).
- **Package name** uses **reverse-domain** naming (e.g. `com.example.hellomobileworld`).
- `res/values/` holds **strings.xml, colors.xml, themes.xml**; `app_name` in strings.xml is the app's display name used by the manifest. Module file: **`build.gradle.kts` (Module: app)**.
- `class MainActivity : AppCompatActivity()`: **AppCompatActivity** is the standard base class for a View-based screen. `super.onCreate(savedInstanceState)` must run first; **`setContentView(R.layout.activity_main)` inflates** the XML and **connects the Kotlin code to the layout**.
- Run = Gradle compiles → packages an **APK** → installs → launches. Kotlin/XML apps have **no Hot Reload**; they must be **rebuilt and reinstalled** (Flutter/React Native support hot reload).
- Kotlin: by JetBrains, **statically typed**, compiles to the **same bytecode as Java**; null safety distinguishes `String?` from `String` at compile time.
- **Troubleshooting:** "SDK location not found" → set SDK path / **`sdk.dir` in local.properties**; black emulator → wait, then **wipe data**; HAXM/VT-x error → enable **virtualization (VT-x/AMD-V) in BIOS**; slow emulator → more RAM, **Hardware GLES 2.0**; Gradle sync failed → internet/firewall, **Sync Project with Gradle Files**; version mismatch → **Upgrade Gradle / Invalidate Caches**; **"Unresolved reference: R"** → **Build → Make Project (Ctrl+F9)** or Gradle sync; **INSTALL_FAILED_INSUFFICIENT_STORAGE** → wipe AVD data; **Manifest merger failed** → `tools:replace`. Always check **Build panel and Logcat** first.

Cross-platform practicals (QuickBite and EventHub)

	Flutter	React Native
Language	**Dart**	**JavaScript / TypeScript** (often with **Expo**)
Check setup	`flutter doctor`	`npx expo --version` (needs **Node.js**)
Navigation library	**Navigator / go_router**	**React Navigation**
State management	**Provider / Riverpod / setState**	**Context API / Redux**
Testing	**Flutter widget tests**	**Jest + React Native Testing Library**
Run on phone	Emulator / USB debugging	**Expo Go** app

- Goal: **one shared codebase** running on **Android and iOS**; responsive layouts; **cart state must persist** between screens (shared app state); transitions within 1 to 2 seconds.
- Version control with **Git and GitHub/GitLab**; testing in the activity was **manual test cases** (Pass/Fail with screenshots) covering navigation, cart logic, validation and responsiveness.
- EventHub requirements show a typical full app: accounts, browse/search/filter, details, booking with validation, **REST API**, **database** (users, events, bookings), **local storage** (session, favourites, preferences) and **notifications** (confirmation, cancellation, reminders).

Week 3 and 4 in-class exercise (case study reminders)

- ** (suggested answers) **FlexiPark** (web-skilled team, one budget, 10-week pilot on Android + iOS, 5-year life, unreliable WiFi, later camera scanning) points to **cross-platform**, with **React Native** suiting a React/JS team; risk: future camera/on-device processing may need **native modules**.
- **PulseFit** (live heart rate, background logging, rotation resets value, testable logic) points to **MVVM**; **Service** for background logging, **Broadcast Receiver** for sensor connect/disconnect, **Activity** for display, **Content Provider** little use unless sharing data; rotation destroys and recreates the Activity, so keep state in a **ViewModel**; **data binding** lets the Stats chart update automatically.